INNOVATION RADAR V2

Requirements vs. Implementation

A gap analysis of the “Innovation Radar — See what's hot for us” briefing against the running Orange Business Innovation Radar V2 application.

Scope	radar_v2/ (Reflex app) + Pipelineteamfile/ (data pipeline)
Method	Direct source reading; live team database queried read-only
Live data	21 real opportunity spaces, 41 articles, 34 classifications
Generated	25 August 2026, 11:29 UTC

How to read the status labels

MET	Built and working as the briefing describes.
PARTIAL	Something exists, but not in the shape the briefing asks for.
MISSING	A stated requirement with no implementation.
NOT REQUIRED	The briefing itself marks it next-phase, illustrative or optional.

Calibration note: the briefing marks slides 15 and 17 as “examples, for inspiration” and slide 18 as next phase. Those are not scored as failures here. Slides 16 and 19 are treated as binding.

SECTION 1

Executive summary

Where the application stands against the briefing, in plain language.

The application delivers the briefing's MVP structure convincingly and its data foundation exceeds what the briefing assumed would exist. Opportunity spaces are modelled exactly as specified — Vertical × Use Case × Technology — over the briefing's own 14 verticals, and the five-stage discovery process from slide 12 runs end to end, automatically, against real institutional sources (TED, CORDIS, OCDS) with a machine-learning noise gate and a live LLM classifier. The briefing describes signal collection as something that "can be done with AI"; the app already does it, on a schedule, with token-cost tracking.

The single biggest gap

The scoring model does not explain itself, and the number it produces is not an attractiveness score. Slide 16 is stated as a hard requirement — "the scoring model must not produce only a number, it must explain the number" — with the explicit test that a user must be able to say why a topic is ranked where it is. Today the headline figure a user sees, labelled "Strategic fit", is this space's article count divided by the largest space's article count. It is a volume proxy with no components, no rationale text and no stored sub-scores. On the current live data that formula gives every topic either 50% or 100%.

The encouraging part is that most of the missing raw material already exists and is simply discarded at the display layer. The classifier stores a genuine per-article confidence and a written justification for every single classification; neither reaches the user interface. Two of the four recommendations that follow are surfacing work, not modelling work.

26 assessed requirements across the briefing

Of the 4 MVP must-haves on slide 19, three are met and one — the attractiveness scoring system — is not.

SECTION 2 · PART 1

Requirement-by-requirement scorecard

What the briefing asks · what the application does today · the code that proves it

REQUIREMENT	WHAT THE BRIEFING ASKS	WHAT THE APPLICATION DOES TODAY	STATUS
MVP technical scope — slide 19 *The briefing's own must-have bar. These four are binding.*			
Opportunity space = Vertical × Use Case × Technology (slides 10, 19)	Spaces must be specific triples, never generic themes such as “AI” or “Cloud”.	Implemented exactly. opportunity_spaces has a UNIQUE constraint on the triple; the taxonomy is closed at 27 use cases and 21 technologies, and the classifier is forbidden from inventing ids. All 21 live spaces are proper triples. `Evidence: collector/storage.py:23-34 · config/taxonomy.json · collector/client.py:113`	MET
A structured, refreshable discovery process (slides 12, 19)	Signals → themes → candidate spaces → curation → scoring → radar, repeatable on demand.	Fully automated in five stages: collect TED/CORDIS/OCDS, balance the corpus, ML noise gate, LLM classification, recompute spaces and write a run summary. Lock file prevents concurrent runs; recompute is idempotent and prunes stale spaces. Triggered from the Refresh page with live progress. `Evidence: Pipelineteamfile/run_radar.py:119-242 · services/pipeline_runner.py`	MET
A visual radar dashboard (slide 19)	A dashboard view of the current opportunity portfolio.	Overview page with portfolio metrics, a relative-fit bar chart and source coverage; Opportunities page with filterable cards; detail page with progress bars and a radial chart. Present and working — though every chart is driven by the volume proxy discussed in section 3. `Evidence: pages/overview.py · pages/opportunities.py · pages/opportunity_detail.py`	MET
An attractiveness scoring system (slide 19)	A scoring system that ranks opportunity spaces by how attractive they are.	No attractiveness model exists. The single ranking number is this space's article count divided by the largest space's article count, displayed as “Strategic fit”. No components are stored or shown. See section 3. `Evidence: services/team_repository.py:87`	MISSING

Requirement-by-requirement scorecard

continued

REQUIREMENT	WHAT THE BRIEFING ASKS	WHAT THE APPLICATION DOES TODAY	STATUS
Scoring must explain itself — slide 16 *Stated as a hard requirement, independent of any formula.*			
The model must explain the number, not only produce it	A user must be able to say why a topic is ranked where it is, or the scoring is not good enough.	Not met. The user sees three percentages and one generated sentence that restates the space's own name and article count. No component breakdown, no source rationale, no explanation of the ranking. The classifier's own written justification exists in the database but is never read by the app. `Evidence: team_repository.py:97 · grep shows evidence column unused in radar_v2/`	MISSING
Attractiveness score present	Each topic carries an attractiveness figure.	A figure labelled “Strategic fit” is present, but it measures corpus volume, not attractiveness. `Evidence: pages/opportunity_detail.py:21`	PARTIAL
Urgency / time horizon	Each topic carries an urgency or time-horizon judgement.	A Now/Next/Later badge exists, but it is a threshold cut on the same volume proxy, not an independent urgency signal. On live data no space can ever fall into “Next”. `Evidence: team_repository.py:95 · components/ui.py:38`	PARTIAL
Source evidence	Visible sources behind each topic.	Strong. Each topic lists its linked institutional records with title, source name, source type, publication date, excerpt and an outbound link to the original. `Evidence: team_repository.py:114-124 · components/ui.py:84`	MET
Signal type	Each signal typed as trend, buying signal, regulation, market move and so on.	No such field exists anywhere in the schema. articles.source_type holds ted / cordis / ocds / rss, which is where the signal came from, not what kind of signal it is. `Evidence: common/models.py · no signal_type field in any table`	MISSING

SECTION 2 · PART 3

Requirement-by-requirement scorecard

continued

REQUIREMENT	WHAT THE BRIEFING ASKS	WHAT THE APPLICATION DOES TODAY	STATUS
"Why hot now", "why this matters", recommended next action as distinct fields	Three separate labelled explanations per topic.	One template sentence covers all three. A "Recommended move" field exists on the detail page but is a hardcoded constant — the same sentence on every topic in the portfolio. `Evidence: pages/opportunity_detail.py:33 (literal string)`	MISSING
User acceptance criteria — slide 13 *Stated as acceptance criteria; note the tension with slide 19.*			
Trust	Every topic explains why it is attractive and urgent, with visible sources.	Sources yes, explanation no. Covered above. `Evidence: see slide 16 rows`	PARTIAL
Freshness	Recent signals plus a last-refresh date.	Signal publication dates are shown on evidence cards, and the Refresh page shows the last run id, pool size, elapsed time and tokens. But last_updated_at is computed per opportunity in team_repository and then never rendered — no page reads the "updated" key. `Evidence: team_repository.py:98 computed · unused in all pages · pages/refresh.py:79-82`	PARTIAL
Actionability	Every topic has a clear next step.	Exceeded in one direction, missing in another. The static "Recommended move" line is not a real next step, but every topic has a one-click "Build business report" that runs a genuine two-stage research and synthesis pipeline into a full decision document. `Evidence: opportunity_detail.py:17 · services/reporting.py:48`	MET
Filtering by vertical	Slice the radar by vertical.	Present as a select populated from the live portfolio, plus free-text search. `Evidence: pages/opportunities.py:25 · state.py:136-143`	MET

SECTION 2 · PART 4

Requirement-by-requirement scorecard

continued

REQUIREMENT	WHAT THE BRIEFING ASKS	WHAT THE APPLICATION DOES TODAY	STATUS
Filtering by geography	Slice the radar by geography.	Not available. Geography exists only as a text attribute of the active company profile, used as prompt context. It is not an attribute of an opportunity space and appears in no filter. `Evidence: extension_store.py:16 · pipeline_runner.py:47`	MISSING
Filtering by business domain	Slice by the six domains: OX Smart Industries, Connectivity, Cybersecurity, Cloud, CX, EX.	The six domains are not modelled as a field anywhere. Vertical and technology are the only structural dimensions. `Evidence: no domain field in taxonomy.json or any schema`	MISSING
Filtering by persona	Slice by target persona; each role mode presets sensible defaults.	No persona concept exists anywhere in the codebase. Confirmed by grep across radar_v2/ and Pipelineteamfile/. `Evidence: zero matches for "persona" outside the word "personalised"`	MISSING
Signal-to-noise	Each view shows only a focused number of topics.	Overview shows the top three; Opportunities is filterable and ordered by relevance then article count. The ML noise gate additionally removes low-signal articles before they can become topics. `Evidence: pages/overview.py:72 · team_repository.py:80 · mlfilter/`	MET
Topic detail depth — slides 5 and 14	*Slide 14 is labelled "AI generated example / inspiration", so it indicates depth rather than a strict spec.*		
Dated, typed signals mixing qualitative and quantitative evidence	Signals carry a date and a type tag.	Dated yes, typed no. See the signal-type row above. `Evidence: articles.published_date present · no type taxonomy`	PARTIAL
Use cases and value drivers with value tags	Where the topic delivers value, tagged by driver.	The use case is named from the closed taxonomy. There are no value drivers and no value tags. `Evidence: config/taxonomy.json`	PARTIAL

Requirement-by-requirement scorecard

continued

REQUIREMENT	WHAT THE BRIEFING ASKS	WHAT THE APPLICATION DOES TODAY	STATUS
Right-to-win: capabilities, proof points, partner ecosystem	Specific internal assets, references and partners per topic.	Not per topic. The Company workspace holds real internal documents and routes their summaries into the classifier prompt, but nothing surfaces on a topic as a named capability, proof point or partner. Slide 18 explicitly scopes this as next phase. `Evidence: pages/company.py · pipeline_runner.company_context_file`	NOT REQUIRED
Numeric score breakdown from sub-criteria bars	Attractiveness, urgency and fit, each built from visible sub-criteria.	Two progress bars and a radial chart, both fed by the same two numbers. No sub-criteria exist to break down. `Evidence: pages/opportunity_detail.py:21-44`	MISSING
Explicitly not required by the briefing			
	Listed by the briefing itself as illustrative, next-phase or optional extra content. Recorded for completeness, not scored as gaps.		
The exact 30/20/20/15/15 weighting (slide 17)	Called "an example, just to inspire you". The compositional shape is the point, not the numbers.	Not implemented, and matching the literal weights should not be a goal. Recommendation 3 addresses the shape instead. `Evidence: n/a — do not treat as contractual`	NOT REQUIRED
CRM-driven right-to-win fine-tuning (slide 18)	Scoped by the briefing as next phase: customer overlap, pipeline value, offering match.	Not built, correctly. The Company workspace is an early, partial move in this direction. `Evidence: pages/company.py`	NOT REQUIRED
Topic watchlist logic (slide 19, extra content)	Keep a topic as an early signal instead of promoting it to a full space.	Not built. Note the app's "priority sources" watchlist is a source list, a different thing. `Evidence: pages/sources.py`	NOT REQUIRED
External signal taxonomy (slide 19, extra content)	trend / regulation / buying signal / market move / technology maturity / proof signal.	Not built. Listed here as optional, but it is also the enabler for the slide-16 signal-typing gap above — worth deciding on together. `Evidence: see recommendation 3`	NOT REQUIRED

SECTION 3

Deep dive: the attractiveness score

What the briefing specifies, what the code computes, and the distance between them.

SPECIFIED (slide 17, illustrative weights)

30%	**Market signal strength**	how visible across external sources
20%	**Source diversity**	does it appear across independent sources
20%	**Evidence quality**	is the signal relevant and credible
15%	**Novelty and momentum**	is the topic emerging or rising
15%	**Strategic relevance**	does it fit the Orange Business context

IMPLEMENTED (team_repository.py:87)

```
relevance = round(
    (avg_client_relevance
     or article_count / max_article_count)
    * 100
)
```

avg_client_relevance is NULL on all 21 live spaces, so the fallback is the only path in practice. One component. No weights. No sub-scores stored anywhere in the schema.

Why avg_client_relevance is always NULL — and it is not a configuration problem

The database schema, the classifier dataclass and the storage layer all carry client_relevance and client_relevance_reason fields, and the application passes a company-context file into every pipeline run it starts. The chain breaks at the prompt: config/prompt_template.txt asks the model to return only {use_case_id, technology_id, confidence, evidence}. It never asks for client_relevance. The parser reads parsed.get("client_relevance") from a response that was never asked to contain it, so the value is always None. Confirmed on the live database: 0 of 34 classifications carry a client_relevance value. This means the Company workspace's “Scoring & fit” document routing currently feeds a prompt that produces no score.

How far the existing data already gets you

Source diversity

Computable today.

COUNT(DISTINCT source_type) and source_name over the linked articles. No model call and no new data — only a column to store the result.

Evidence quality

Computable today.

AVG(article_classifications.confidence) over the linked articles. Populated on all 34 rows and already joined one line away in the detail query.

Novelty and momentum

Needs a small change.

articles.published_date is stored but feeds no score. Comparing recent signals against older ones turns momentum into a real measurement.

Signal strength and relevance

Needs a decision.

The first needs a definition of “visibility” you agree on; the second needs the prompt fix above. These two are the genuine design work.

SECTION 3 · CONTINUED

One real topic, in the app right now

Opportunity space #3 as the interface presents it, next to the evidence underneath it.

Defense · Compliance monitoring × Cybersecurity Platform

Last updated 2026-08-25 · opportunity_spaces id = 3 · 2 linked articles

Strategic fit	Evidence confidence	Supporting signals	Momentum
100%	**56%**	**2**	**+4%**
= 2 / 2 articles	= 48 + 2 × 4	linked records	= 2 × 2, capped

Everything the interface says about why this topic is here:

> *"2 institutional signals connect Compliance monitoring with Cybersecurity Platform in Defense."*

That sentence is a template. It restates the space's own name and its article count. It is the only rationale the user gets.

What the pipeline actually knows about this topic — and does not show

Real classifier confidence	0.93 and 0.84 — mean 89%, not the 56% displayed.
Written justification, article 5	"The article focuses on cybersecurity certification and assessment tools for meeting CSA and CRA requirements, aligning with continuous regulatory compliance verification…"
Source diversity	Both articles are CORDIS. One source type, one source name. The briefing's second scoring dimension would penalise this heavily; the app has no field to express it.
Signal recency	Published 2026-01-01 and 2026-09-01. Dates are stored and shown on evidence cards, but never feed novelty, momentum or the time horizon.

The written justification above is stored in article_classifications.evidence for all 34 classifications. A grep across radar_v2/ shows it is never selected: team_repository.opportunity_detail reads articles.summary instead. The rationale slide 16 demands already exists, unused.

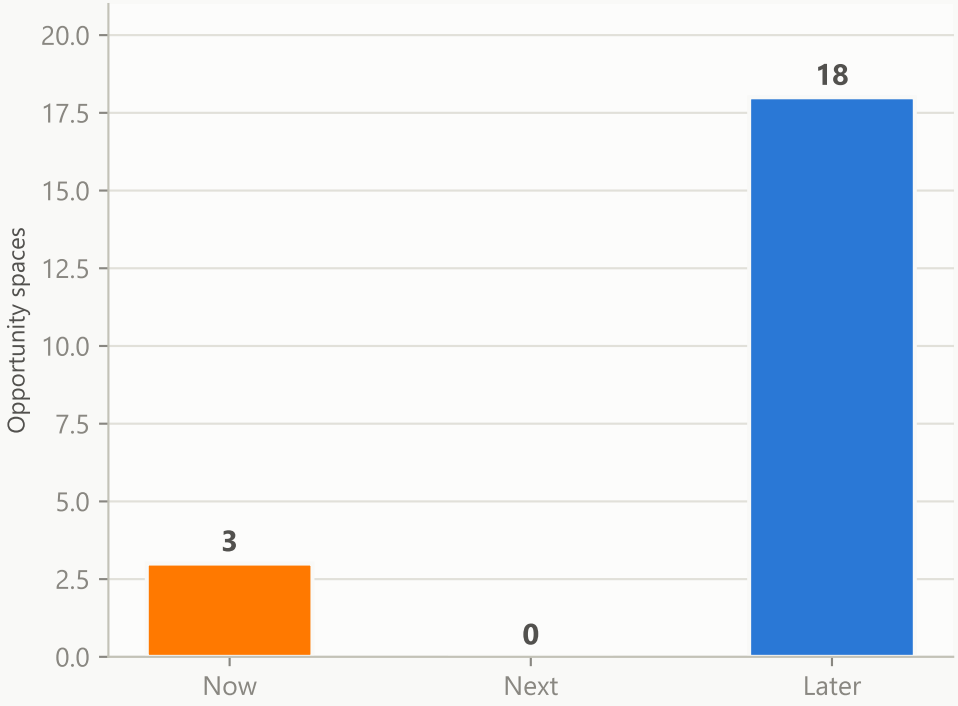
The knock-on effect on time horizon

Now / Next / Later is a threshold cut on the same volume proxy, so it inherits its behaviour.

```
horizon = "Now" if relevance >= 82 else "Next" if relevance >= 68 else "Later"
```

Because relevance is article_count divided by the largest article_count, and the largest space on the live database holds 2 articles, relevance can only take the values 50 or 100. Every threshold in between is unreachable. The chart below is the actual distribution of all 21 spaces in the database today.

Time horizon across the 21 live spaces



What this means in the interface

The “Next” option in the horizon filter on the Opportunities page returns nothing at all today.

Three topics are labelled “Now” — signalling urgency to a salesperson — purely because they have two linked documents instead of one.

The label is not wrong by a small margin. It is measuring a different thing from the one the briefing describes: urgency and the closing of a market window, rather than corpus volume.

A fair caveat: this is a young corpus. With thousands of articles the ratio would spread out and the buckets would fill. But the underlying objection survives the data volume — a topic covered by many documents is not thereby urgent, and ranking by relative corpus share will always reward whichever sources the collectors happen to cover most densely.

SECTION 4

What the application does beyond the briefing

Real, working functionality the briefing never asks for. None of this is scored above.

Company workspace

`radar_v2/pages/company.py · services/knowledge.py`

Upload company documents, summarise them individually or as one combined report through the LLM, then route each summary to a named destination — Opportunity mapping, Scoring & fit, Business reports or Everywhere — or switch it off. This is a concrete implementation of the briefing's slide-18 “next phase” right-to-win idea, arriving early. It reaches the classifier as prompt context; it does not yet reach the score.

Two-stage research reports

`radar_v2/services/reporting.py`

Per opportunity, the app plans varied research queries with an LLM, runs them through a live web search provider (Tavily or SearXNG), then synthesises a decision document: executive summary, market signal, financial indicators, company fit, competitor and partner landscape, risks with likelihood and impact, a phased roadmap, a recommendation and numbered sources. Persisted and re-openable. The briefing asks for a “next step”; this is a full business case.

Focused discovery

`radar_v2/pages/discovery.py`

Ask a free-text market question, search the web, then promote chosen results into the pipeline as articles with source_type="web_discovery" so they pass the same ML gate and classifier as institutional evidence. The single-pipeline boundary is respected — discovery never writes classifications directly.

Priority source watchlist

`radar_v2/pages/sources.py · extension_store.fetch_custom_source_articles`

Register custom source URLs, fetch a readable record from each, and feed them into the next radar cycle. Note this is a source watchlist, not the topic watchlist listed as optional extra content on slide 19.

Operational cost transparency

`run_radar.py · article_classifications.tokens_used`

Every classification records its token usage; every run sums the tokens spent and writes a JSON summary with elapsed time, per-source collection counts and the ML gate's keep/delete split. 56,027 tokens tracked so far. The briefing never mentions cost; someone will be glad this exists.

Session-only credential handling

`radar_v2/pages/settings.py · state.activate_provider`

Provider base URL, model and mode are persisted; the API key is activated for the session only and never written to disk. A sensible default the briefing does not specify.

SECTION 5

Recommendations

Ordered by value returned per unit of work. Items 1 and 2 are surfacing, not modelling.

1 Show the confidence the classifier actually produced

Small · half a day

Replace the synthetic confidence = min(96, 48 + article_count * 4) in team_repository.list_opportunities with the mean of article_classifications.confidence across the space's linked articles. The column is populated on all 34 rows and already read one join away in opportunity_detail. On space #3 this changes a displayed 56% into a defensible 89%. This is the single highest-value change in the report.

2 Surface the rationale the model already wrote

Small · one day

article_classifications.evidence holds a written, article-grounded justification for every classification and is never selected by the app. Add it to the evidence query and render it on the detail page. Then split the one generated sentence into the distinct labelled fields slide 16 names — why hot now, why this matters, recommended next action — rather than a single template. Note that "Recommended move" on opportunity_detail.py:33 is currently a hardcoded constant string, identical on every topic.

3 Decide what "attractiveness" should mean, then store it as components

Medium · the real design work

Do not chase slide 17's exact weights — the briefing calls them illustrative. Do adopt the shape: a small number of named sub-scores, each persisted, that sum to the headline. Two of the five are computable today from existing data with no new LLM calls: source diversity (distinct source_type and source_name across linked articles) and evidence quality (mean classifier confidence). Novelty and momentum become real once published_date is used instead of article_count. Strategic relevance is the one that needs the prompt fix below. Store the components in opportunity_spaces so the interface can draw the breakdown bars from slide 14.

4 Close the client_relevance dead path, or remove it

Small · one line of prompt, then re-run

prompt_template.txt does not ask the model for client_relevance or client_relevance_reason, so both columns are permanently NULL and the Company workspace's "Scoring & fit" routing has no effect on any number. Either add both keys to the requested JSON object and the accompanying rule text, or delete the columns and the fallback branch so nobody later assumes a score is being computed. Leaving it half-wired is the worst of the three options.

One open question for you, flagged rather than decided

The briefing contradicts itself on persona and geography filtering. Slide 13's acceptance criteria say users must be able to slice by vertical, geography, domain and persona, and slide 5 asks for persona presets. But slide 19's MVP technical scope lists only four must-haves, none of which mention filtering beyond a visual radar dashboard. The app implements neither persona nor a filterable geography, and the six business domains from slide 6 are not modelled as a field. That is three distinct build efforts. This report does not call them failures, because the MVP slide does not require them — but it is your call, not a coding decision, and it should be made before anyone starts. Ask the briefing's author which slide governs.